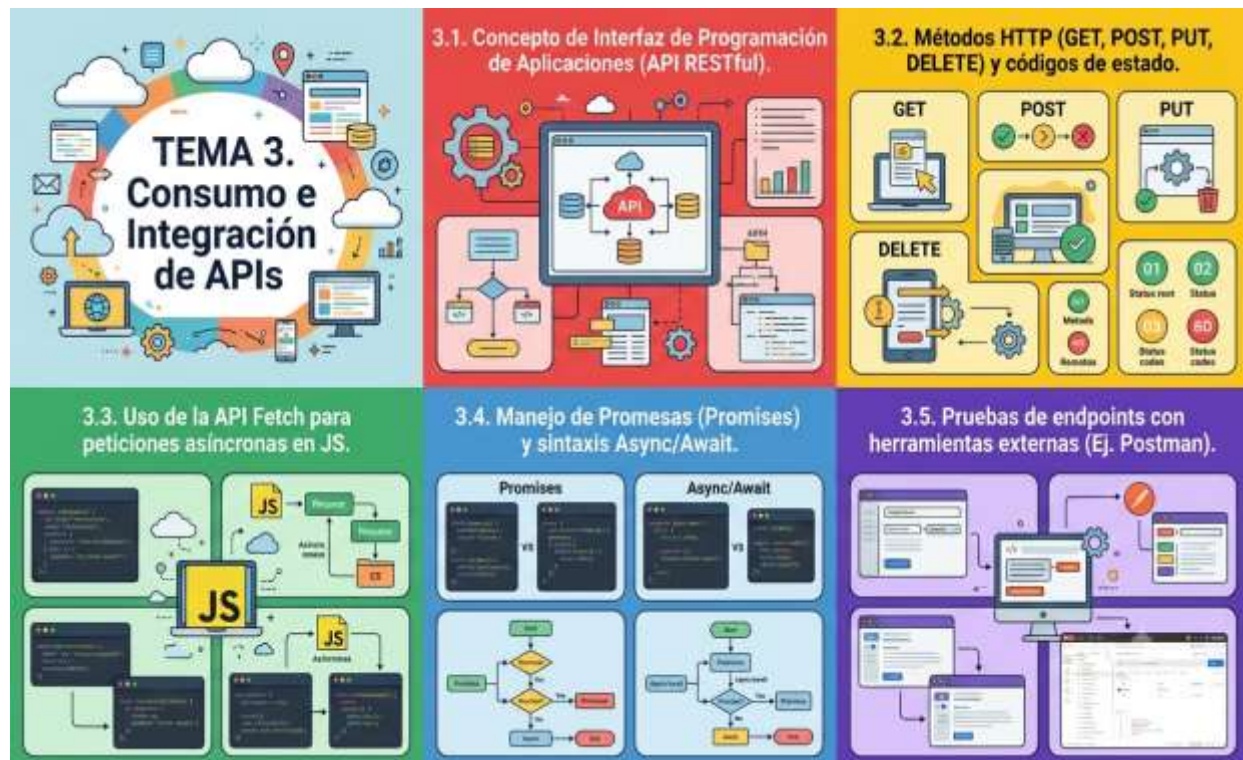


TEMA N° 3



TEMA 3 CONSUMO E INTEGRACIÓN DE APIS



EN ESTE TEMA SERÁS CAPAZ DE...

1. **Comprender** la arquitectura y el funcionamiento de una API RESTful, identificando cómo interactúan los sistemas web modernos a través de peticiones y respuestas.
2. **Desarrollar** la capacidad de consumir servicios externos utilizando la API Fetch y asincronismo (Async/Await) en JavaScript para enriquecer tus aplicaciones web.
3. **Probar y validar** el correcto funcionamiento de los endpoints utilizando herramientas profesionales como Postman, garantizando integraciones robustas como Técnico en Sistemas Informáticos.



PRÁCTICA



¿Alguna vez te has preguntado cómo la aplicación del clima en tu celular sabe exactamente si va a llover en Pailón hoy a las 9 de la mañana?

Imagina la siguiente situación en nuestro Centro de Educación Alternativa (CEA) Herman Ortiz Camargo:

Gracias al esfuerzo de todos, acabamos de estrenar el nuevo y bien equipado módulo educativo en el Barrio Saó. La dirección quiere instalar una pantalla informativa en el ingreso principal. Esta pantalla debe mostrar la siguiente información en tiempo real:

1. Las noticias de la alcaldía.
2. El horario de llegada de los trenes desde la estación del Barrio 24 de Septiembre.
3. Los resultados de los torneos deportivos que se juegan en el coliseo del Barrio 27 de Mayo.
4. El clima actual para que los estudiantes que vienen caminando desde el Barrio San Expedito o Jardines de Pailón sepan si necesitan paraguas.

Tú, como Técnico en Sistemas Informáticos encargado del proyecto, te enfrentas a un reto: la información está dispersa. El CEA no controla los trenes, ni el clima, ni los partidos del coliseo. Entonces, ¿cómo logramos que nuestra pantalla obtenga todos esos datos automáticamente sin tener que escribirlos a mano cada hora? La respuesta está en lograr que nuestros sistemas "hablen" con los sistemas de ellos.



TEORÍA



3.1. Concepto de Interfaz de Programación de Aplicaciones (API RESTful)

Una **API** (Application Programming Interface) es un puente de comunicación. Es un conjunto de reglas que permite que dos aplicaciones de software distintas se comuniquen entre sí. Piensa en una API como si fuera un mesero en un restaurante del Barrio Central Fátima. Tú (el



cliente/aplicación web) miras el menú y le pides al mesero (la API) una sopa de maní. El mesero va a la cocina (el servidor/base de datos), pide la sopa, espera a que esté lista y te la trae a la mesa. Tú no necesitas saber cómo encendieron la cocina o qué receta usaron; solo haces el pedido y recibes el resultado.



Cuando le agregamos el término **RESTful**, nos referimos a una API que sigue las reglas de la arquitectura REST (Representational State Transfer). Estas reglas dicen que la comunicación debe hacerse a través de la web (usando el protocolo HTTP) y que la información debe viajar en un formato ligero y fácil de leer, que hoy en día es casi siempre el formato **JSON** (JavaScript Object Notation).

3.2. Métodos HTTP (GET, POST, PUT, DELETE) y códigos de estado

Para que nuestro "mesero" (la API) entienda qué queremos hacer con los datos, utilizamos los **Métodos HTTP**. Son los verbos de la web:

1. **GET (Obtener):** Solo pide información. *Ejemplo:* Pedir la lista de estudiantes inscritos que viven en el Barrio Residencial Norte.
2. **POST (Crear):** Envía información nueva al servidor. *Ejemplo:* Registrar un nuevo estudiante del Barrio María Belén en la base de datos del CEA.



3. **PUT (Actualizar):** Modifica una información que ya existe. *Ejemplo:* Actualizar la dirección de un estudiante que se mudó del Barrio Oto Waver al Barrio San Antonio.
4. **DELETE (Eliminar):** Borra un registro. *Ejemplo:* Eliminar un evento deportivo cancelado en el coliseo municipal.

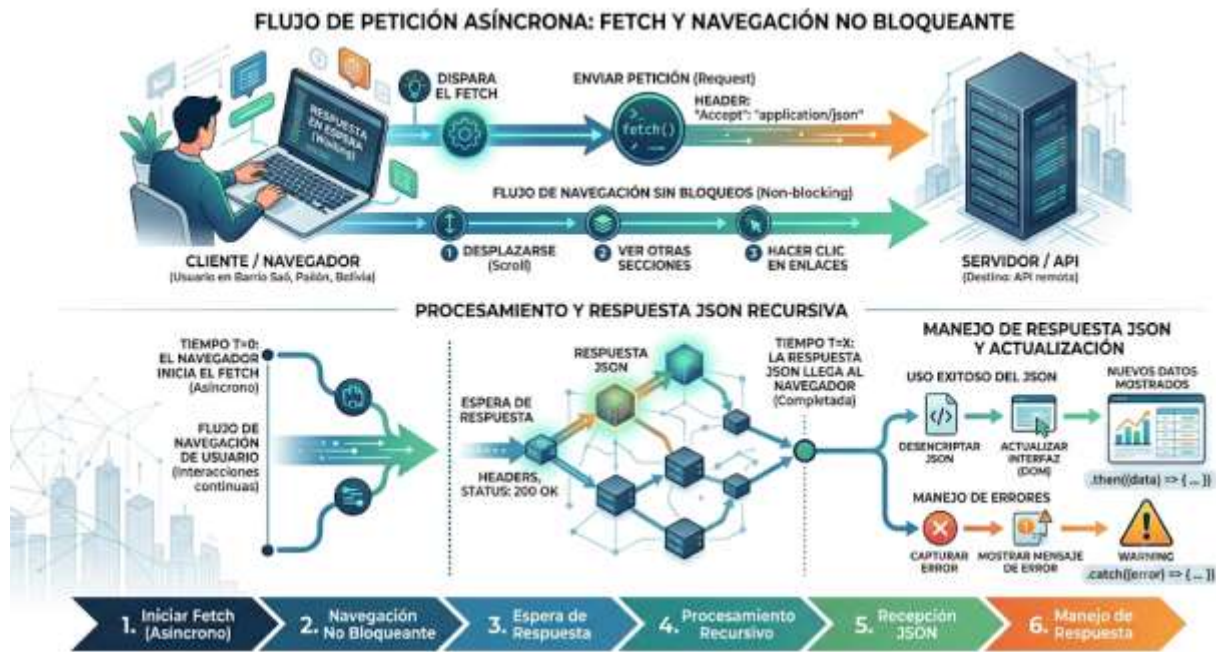
Códigos de Estado: Cuando el servidor responde, no solo envía datos, sino un número (código) para decirte cómo salió todo:

1. **200 (OK):** Todo salió perfecto.
2. **201 (Created):** El POST funcionó y se creó el registro.
3. **400 (Bad Request):** Te equivocaste al enviar los datos (ej. faltaba el nombre).
4. **404 (Not Found):** Lo que buscas no existe (el famoso error de página no encontrada).
5. **500 (Internal Server Error):** El servidor falló o colapsó.

3.3. Uso de la API Fetch para peticiones asíncronas en JS

Como Técnico en Sistemas Informáticos, tu herramienta principal en la web será JavaScript. Para comunicarte con una API desde JavaScript, usamos la función nativa llamada **fetch()**.

El término "asíncrono" es clave aquí. Cuando haces una petición a un servidor de internet, la respuesta no es inmediata. Tarda unos milisegundos o segundos. Si JavaScript se detuviera a esperar, toda tu página web se congelaría. La programación asíncrona permite que JavaScript diga: "*Ve a buscar estos datos y, **mientras tanto**, yo seguiré cargando el resto de la página. Avísame cuando termines*".



3.4. Manejo de Promesas (Promises) y sintaxis Async/Await

`fetch()` devuelve algo llamado **Promesa (Promise)**. Una Promesa en JavaScript es exactamente igual que en la vida real: representa un valor que puede estar disponible ahora, en el futuro, o nunca. Una Promesa tiene tres estados:

1. *Pendiente (Pending)*: La petición viajando por internet.
2. *Cumplida (Fulfilled)*: Los datos llegaron bien.
3. *Rechazada (Rejected)*: Se cortó el internet o el servidor falló.

Aunque antiguamente se usaban las palabras `.then()` y `.catch()` para manejar promesas, hoy en día usamos la sintaxis moderna **async / await**, que hace que el código asíncrono se lea como si fuera código síncrono normal.

JavaScript

```
// Definimos una función ASÍNCRONA usando la palabra clave '
async'
async function obtenerClimaPailon() {
  // Usamos '
  try...catch
```



```

' para capturar cualquier error de red    try {
// '
await
' pausa la ejecución de esta línea hasta que la API responda    // Hacemos
un GET (por defecto) a una API imaginaria del clima    const respuesta =
await fetch('
https:
//api.clima.com/pailon');
// Verificamos si el código de estado no es un error (ej. 404 o 500)    if
(!respuesta.ok) {
// Si hay error, lanzamos una excepción    throw new Error('
Error en la conexión con el servidor del clima');
}
// '
await
' vuelve a esperar mientras el texto recibido se convierte en un objeto JSON
const datos = await respuesta.json();
// Imprimimos los datos en la consola    console.log("
El clima en Pailón es:", datos.temperatura);
}
catch (error) {
// Si el internet falla o la API cae, el error se captura aquí
console.error("
Hubo un problema:", error);
}
}
// Llamamos a la función obtenerClimaPailon();

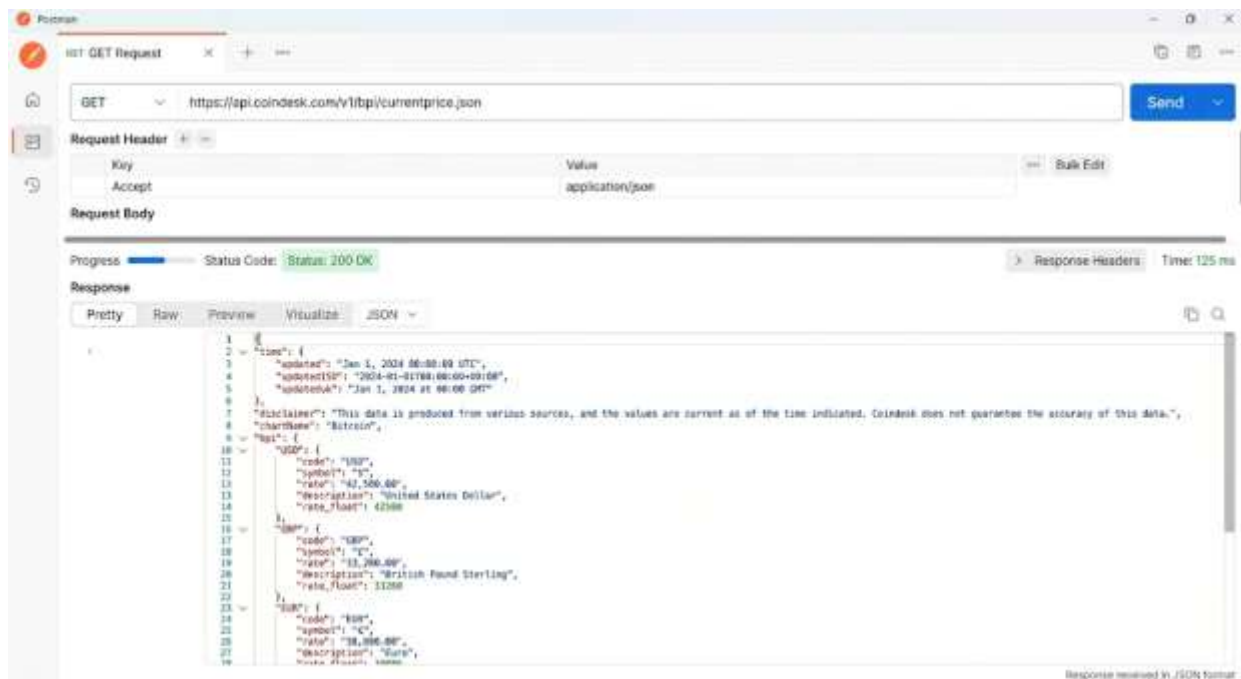
```

3.5. Pruebas de endpoints con herramientas externas (Ej. Postman)

Antes de escribir código JavaScript, un buen Técnico prueba que la API funcione correctamente. Imagina programar por horas solo para descubrir que la API de los trenes del Barrio 24 de Septiembre estaba apagada.

Un **Endpoint** es simplemente la URL específica a la que hacemos la petición (ej. <https://miapp.com/api/estudiantes>).

Para probar estos endpoints usamos herramientas como **Postman** o **Thunder Client**. Estos programas ofrecen una interfaz gráfica donde pones la URL, seleccionas el método (GET, POST), presionas "Send" (Enviar) y ves inmediatamente qué responde el servidor y qué código de estado arroja.



3.6. Consumo de APIs de Inteligencia Artificial (OpenAI / Gemini)

En la actualidad, las APIs no solo sirven para consultar bases de datos. Una de las tendencias más fuertes es integrar Inteligencia Artificial en nuestros sistemas. Por ejemplo, utilizando la API de Google Gemini o de OpenAI, podrías crear una plataforma para el CEA Herman Ortiz Camargo donde el estudiante ingrese una duda sobre Sistemas Operativos, y tu sistema envíe esa duda a la API de IA, reciba la explicación pedagógica, y se la muestre al alumno. Todo el "cerebro" está en la nube, tú solo gestionas el puente (API).

3.7. Autenticación y Seguridad en APIs (Tokens y JWT)

Si abres un endpoint tipo DELETE /api/estudiantes, ¡cualquiera podría borrar la base de datos del CEA! Por ello, las APIs públicas de lectura son raras. La mayoría requiere autenticación. El estándar actual de la industria es el uso de **JSON Web Tokens (JWT)**. Cuando el administrador del sistema inicia sesión con su usuario y contraseña, el servidor le devuelve un Token (una cadena de texto larga y encriptada). En cada petición Fetch posterior, el JavaScript debe incluir ese Token como una "credencial" en los encabezados HTTP para demostrar que tiene permiso de ejecutar esa acción.



3.8. Webhooks y Arquitecturas Basadas en Eventos

Normalmente, el cliente siempre inicia la comunicación (haciendo un GET o un POST). Pero, ¿qué pasa si queremos saber el instante exacto en que ocurre algo? Si quisiéramos saber cuándo se inscribe un nuevo estudiante del Barrio Ovidio Barberi, hacer un Fetch cada minuto (Polling) sobrecarga el servidor.

La tecnología moderna utiliza **Webhooks**, que son "APIs invertidas". En lugar de que tú preguntes a cada rato, tú le das al servidor una URL tuya y le dices: *"No me llames, yo te llamo... Bueno, al revés: Cuando se inscriba alguien, mándame un POST a mi URL"*. Esto optimiza el rendimiento y es la base de las arquitecturas de eventos modernas.

CIERRE TEÓRICO

❑ Mitos vs. Realidades

Mito Común	La Realidad Técnica
"Las APIs solo sirven para desarrollo web avanzado."	Falso. Hoy en día, desde aplicaciones móviles, videojuegos, hasta sistemas de domótica y sensores IoT consumen APIs constantemente.
"Hacer una petición a una API expone mis datos privados."	Falso. Si la API utiliza el protocolo HTTPS y buenos métodos de autenticación, la información viaja cifrada y segura de extremo a extremo.

❑ Glosario de Términos

1. **Endpoint:** Una ruta o URL específica de un servidor a la que nos conectamos para interactuar con un recurso particular de la API.
2. **JSON (JavaScript Object Notation):** Formato de texto ligero y universal utilizado para intercambiar datos entre el cliente y el servidor.
3. **Payload:** El "cuerpo" del mensaje. Son los datos útiles que se envían en una petición POST o PUT al servidor.
4. **Asíncrono:** Procesos que no bloquean el hilo principal de ejecución; permiten que el programa siga trabajando mientras espera una respuesta lenta (como una petición de red).



5. **API Key:** Un código alfanumérico que actúa como una llave secreta que el servidor usa para identificar y autorizar al programa que está realizando la petición.

▣ *Ponte a Prueba*

1. ¿Qué método HTTP se utiliza estrictamente para obtener datos sin modificarlos?

1. a) POST
2. b) GET
3. c) PUT

2. ¿Qué código de estado indica que el servidor no encontró el recurso solicitado?

1. a) 200 OK
2. b) 500 Internal Server Error
3. c) 404 Not Found

3. En JavaScript, ¿qué palabra clave se usa para pausar la ejecución de una función asíncrona hasta que se resuelva la Promesa?

1. a) await
2. b) async
3. c) fetch



VALORACIÓN



Al integrar APIs en nuestros sistemas, no solo conectamos computadoras; estamos manejando información del mundo real que impacta vidas. Como Técnicos en Sistemas Informáticos, debemos reflexionar sobre la **Privacidad de la Información**. Si desarrollamos un sistema para el CEA Herman Ortiz Camargo y consumimos APIs para registrar la ubicación de nuestros estudiantes (por ejemplo, saber si vienen del Barrio Las Misiones o San José), tenemos una responsabilidad ética profunda. Esos datos no deben ser expuestos ni vendidos a terceros.



Además, debemos ser conscientes del **Impacto Medioambiental (e-waste y consumo energético)**. Cada vez que nuestro código ejecuta un ciclo asíncrono ineficiente o hace peticiones innecesarias (ej. consultar el clima cada 2 segundos), estamos obligando a los servidores (que son grandes centros de datos físicos) a consumir más energía eléctrica y generar mayor huella de carbono. Optimizar nuestras peticiones API, usar cachés y solicitar solo los datos necesarios no solo mejora la velocidad de nuestra app, sino que es una práctica de desarrollo sostenible que protege nuestro entorno natural en la región chiquitana.



PRODUCCIÓN



Reto Final: Panel Informativo para el CEA

Vamos a construir un pequeño programa en tu computadora para practicar el consumo de una API simulada.

Paso 1: Preparar el entorno HTML

Crea un archivo llamado index.html en tu computadora. En este archivo pondremos un botón y un espacio para mostrar los datos.

HTML

```
<!DOCTYPE html>
<html lang="
es">
<head>   <meta charset="
UTF-8">   <title>Panel del CEA Herman Ortiz Camargo</title>
</head>
<body>   <h1>Estación de Tren - Barrio 24 de Septiembre</h1>           <button
onclick="
obtenerHorarios()">Ver estado del tren de las 9 de la mañana</button>
<div id="
resultado" style="
margin-top: 20px;
font-weight: bold;
color: blue;
">
</div>   <script src="
app.js">
```



```
</script>
</body>
</html>
```

Paso 2: Escribir la lógica en JavaScript

Crea un archivo llamado app.js en la misma carpeta. Vamos a usar la API pública *JSONPlaceholder* para simular que recibimos un aviso del tren.

JavaScript

```
// Función asíncrona enlazada al botón del HTML async function obtenerHorarios()
{
// 1. Obtenemos la referencia del contenedor en el HTML    const contenedor =
document.getElementById('
resultado');
// 2. Mostramos un mensaje temporal de carga    contenedor.innerText = "
Consultando con la estación del Barrio 24 de Septiembre...";
try {
// 3. Hacemos la petición GET a una API de pruebas gratuita    // Usamos
await para esperar que el servidor responda    const respuesta = await
fetch('
https:
//jsonplaceholder.typicode.com/posts/1');
// 4. Verificamos si hubo un problema de conexión    if (!respuesta.ok) {
throw new Error(`Error HTTP: {
respuesta.status}
`);
}
// 5. Convertimos la respuesta cruda a formato JSON    const datos = await
respuesta.json();
// 6. Mostramos la información en pantalla    // (Simulamos que '
title
' es el reporte del tren)    contenedor.innerText = `Reporte Recibido: {
datos.title}
`;
}
catch (error) {
// 7. Si hay error (ej. sin internet), mostramos un mensaje amigable
contenedor.innerText = "
Error al conectar con la estación. Intente de nuevo.";
console.error("
Detalle del error:", error);
}
}
```

Paso 3: Ejecución



Abre tu archivo index.html con cualquier navegador web (como Chrome o Edge). Haz clic en el botón y observa cómo tu página solicita datos externos y los muestra en la pantalla sin necesidad de recargar la página completa. ¡Acabas de consumir tu primera API como Técnico!

5. RECURSOS ADICIONALES

Para profundizar tus conocimientos y convertirte en un experto desarrollando integraciones en la web, consulta las siguientes herramientas y documentaciones oficiales:

1. **MDN Web Docs (Uso de Fetch):** La guía definitiva y oficial sobre cómo utilizar la API Fetch en JavaScript, mantenida por Mozilla. Ideal para ver todos los parámetros de configuración (headers, body). developer.mozilla.org/es/docs/Web/API/Fetch_API/Using_Fetch
2. **Postman Learning Center:** Documentación oficial para aprender a descargar, instalar y probar tus endpoints con Postman antes de tirar una sola línea de código. learning.postman.com
3. **JSONPlaceholder:** Una "Fake REST API" gratuita online que te sirve para hacer pruebas y prototipos de tus aplicaciones con peticiones GET, POST, PUT y DELETE sin tener que configurar tu propio servidor primero. jsonplaceholder.typicode.com